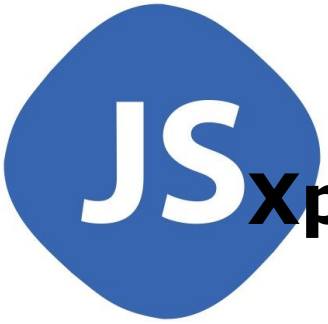


Хранение данных в браузере

- Введение
- LocalStorage
- SessionStorage
- Перебор ключей
- Событие Storage
- Послесловие

Введение

Объекты веб-хранилища **localStorage** и **sessionStorage** позволяют хранить пары ключ/значение в браузере. Объекты являются ключевым элементом при разработке асинхронных веб-приложений.

Что в них важно – данные, которые в них записаны, сохраняются после обновления страницы (в случае sessionStorage) и даже после перезапуска браузера (при использовании localStorage).

> ***Примечание***. Объекты **Storage** это полезный инструмент веб-разработчика и источник раздражения пользователя. Если вас не раздражает контекстная реклама, тогда вы дважды в плюсе ;)

Но ведь у нас уже есть куки. Зачем тогда нужны объекты веб-хранилища?

> ***Примечание***. **Куки** (cookie) – это файлы с информацией, полученной при посещении пользователем **веб-ресурса**. Файлы хранятся на жестком диске компьютера пользователя. В файлах, как правило, отображаются ваши предпочтения, наиболее часто посещаемые тематики, логины и пароли.
>
> **Каждый раз**, когда вы посещаете сайт, который использует куки, браузер отправляет эти данные на сервер для того, чтобы правильно опознать вас

среди других пользователей. Куки бывают временными и постоянными.

Про файлы куки будем говорить отдельно в одной из следующих тем.

- Объекты веб-хранилища не отправляются на сервер при каждом запросе. Именно поэтому мы можем хранить гораздо больше данных. Большинство современных браузеров могут выделить как минимум 5 мегабайтов данных (или больше), и этот размер можно поменять в настройках.
- Сервер не может манипулировать объектами хранилища через HTTP-заголовки. Всё делается при помощи JavaScript.
- Хранилище привязано к источнику (**домен**/**протокол**/**порт**). Это значит, что разные протоколы или поддомены определяют разные объекты хранилища, и они не могут получить доступ к данным друг друга.

Объекты хранилища **localStorage** и **sessionStorage** предоставляют одинаковые методы и свойства:

- **setItem**(key, value) – сохранить пару ключ/значение.
- **getItem**(key) – получить данные по ключу key.
- **removeItem**(key) – удалить данные с ключом key.
- **clear**() – удалить всё.
- **key**(index) – получить ключ на заданной позиции.
- **length** – количество элементов в хранилище.

Интерфейс объектов **Storage** похож на **Map** (setItem/getItem/removeItem), но также позволяет получить **доступ к элементу по индексу** – **key**(index).

Протестируем работу веб-хранилищ на примерах.

localStorage

Основные особенности объекта **localStorage**:

- Этот объект **один на все вкладки и окна** в рамках источника (один и тот же домен/протокол/порт).

- Данные **не имеют срока давности**, по которому истекают и удаляются. Сохраняются после перезапуска браузера и даже ОС.

Например, если запустить следующий код:

```
localStorage.setItem('key', 'test');
```

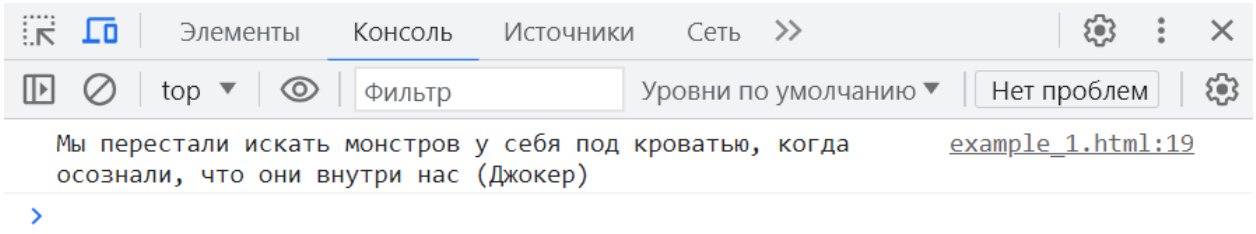
…и закрыть/открыть браузер или открыть ту же страницу в другом окне, то можно получить данные следующим образом:

```
alert (localStorage.getItem('key')); // test
```

example_1. Сохранение в веб-хранилище строковых данных

```
<script>
    let test = "Мы перестали искать монстров у себя под кроватью, 
    когда осознали, что они внутри нас (Джокер)";
    // сохраняем в localStorage строку
    window.localStorage.setItem('key', test);
    // получаем значение localStorage по ключу
    console.log(window.localStorage.getItem('key'));
</script>
```

Получили доступ к сохраненной в хранилище строке.

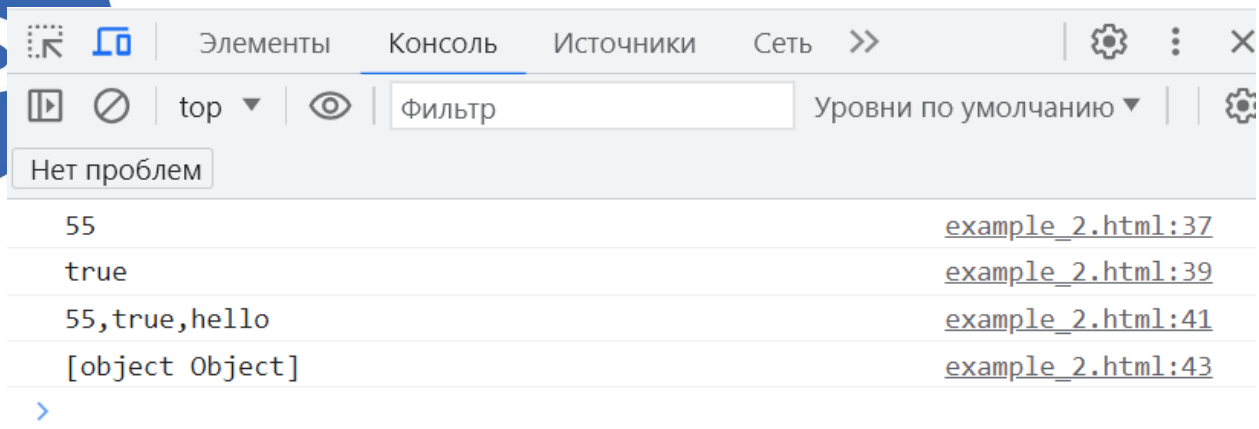


В объектах **Storage** можно хранить не только строковые данные. Хотя, есть нюансы. В демонстрационном примере **example_2** продемонстрирована возможность сохранения в объект хранилища данных разных типов. Обратите внимание на вывод этих данных в консоль браузера.

example_2. Сохранение в веб-хранилище данных разных типов

```
<script>
    // сохраняем в localStorage число
    window.localStorage.setItem("num", 55);
    // сохраняем в localStorage булево значение
    window.localStorage.setItem("bol", true);
    // сохраняем в localStorage массив
    window.localStorage.setItem("arr", [55, true, "hello"]);
    // сохраняем в localStorage объект
    window.localStorage.setItem("obj", { num: 255, bool:true,
    msg:"hello" });
    // извлекаем из localStorage данные
    let num = localStorage.getItem('num');
    let bol = localStorage.getItem('bol');
    let arr = localStorage.getItem('arr');
    let obj = localStorage.getItem('obj');
    // выводим данные в консоль браузера
    console.log(num);
    console.log(bol);
    // ! обратите внимание, как выводится массив
    console.log(arr);
    // ! обратите внимание, как выводится объект
    console.log(obj);
</script>
```

Данные из хранилища выводятся в виде **строк**.



Веб хранилища позволяют хранить структуры любой сложности.

Объект веб-хранилища хранит только строки

Ключ и значение объекта хранилища должны быть **строками**. Если мы используем любой другой тип, например число, массив или объект, то он **автоматически преобразуется в строку**.

А это значит, что если необходимо сохранить сложный тип данных, то предварительно его необходимо сериализовать. Чаще всего для этого вы будете использовать формат **JSON-представления** данных.

example_3. Сериализуем данные для хранения

```
<script>
    // объект для сохранения в localStorage
    let user = {
        name: "Евгений",
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Ленинград",
            post: 101101
```

```
        },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
        ]
    };
    // просто сохранить объект не получится
    // объект необходимо перевести в формат JSON
    let userJSON = JSON.stringify(user);
    // сохраняем объект JSON в localStorage
    window.localStorage.setItem("user", userJSON);
    // проверим значение localStorage
    console.log(localStorage.getItem("user"));
</script>
```

Также возможно привести к строке весь объект хранилища, например для отладки:

```
JSON.stringify(localStorage);
```

Доступ как к обычному объекту

Также можно получать/записывать данные в объект хранилища, как в обычный **JS-объект**. Это возможно по историческим причинам.

```
// установить значение для ключа
localStorage.test = 2;
// получить значение по ключу
alert( localStorage.test ); // 2
// удалить ключ
```

```
delete localStorage.test;
```

example_4. Веб-хранилище — это обычный объект

```
<script>
    // сложная структура данных для хранения
    let user = {
        name: "Евгений",
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Ленинград",
            post: 101101
        },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
        ]
    };
    // предварительно очистим объект
    localStorage.clear();
    // доступ как к обычному объекту
    localStorage.num = 100;
    localStorage.str = "Добро пожаловать!";
    localStorage.bool = true;
    localStorage.arr = ["js", "шпрехшталмейстер", "балалайка"];
    localStorage.obj = user;
    // вывод свойств объекта в консоль
```

```
    console.log(localStorage.num);
    console.log(localStorage.str);
    console.log(localStorage.bool);
    console.log(localStorage.arr);
    console.log(localStorage.obj);
</script>
```

И если вы решили использовать хранилище как обычный объект, принцип хранения не меняется – **значение свойства объекта веб-хранилища должно быть строкой**.

example_5. Значение веб-хранилища всегда строка

```
<script>
    // сложная структура данных для хранения
    let user = { /* ... */ };
    // предварительно очистим объект
    localStorage.clear();
    // доступ как к обычному объекту
    localStorage.num = 100;
    localStorage.str = "Добро пожаловать!";
    localStorage.bool = true;
    localStorage.arr = JSON.stringify(["js", "шпрехшталмейстер",
    "балалайка"]);
    localStorage.obj = JSON.stringify(user);
    // вывод свойств объекта в консоль
    console.log(localStorage.num);
    console.log(localStorage.str);
    console.log(localStorage.bool);
```

```
    console.log(JSON.parse(localStorage.arr));
    console.log(JSON.parse(localStorage.obj));
</script>
```

И несмотря на то, что как правило, это работает, делать так **обычно не рекомендуется**.

- Если ключ генерируется пользователем, то он может быть каким угодно, включая **length** или **toString** или другой встроенный метод **localStorage**. В этом случае **getItem**/**setItem** сработают нормально, а вот чтение/запись как свойства объекта не пройдут.
- Когда мы модифицируем данные, то срабатывает событие **storage**. Но это событие не происходит при записи без **setItem**, как свойства объекта.
- И вообще, зачем веб-хранилище использовать как обычный объект. Хранилище — это хранилище, а объект – это... (ну вы поняли).

Примечание. Объекты и события рассматриваются в отдельных темах.

Домен / протокол / порт

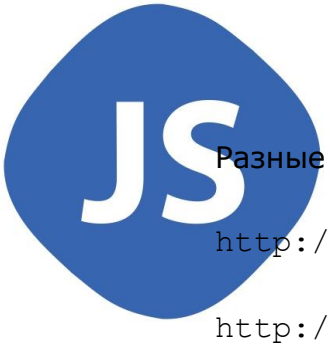
Нет никакого смысла сохранять данные в объект **Storage** и тут же его извлекать. Есть смысл (смыслище) сохранить объект на одной странице, для использования данных этого объекта на другой странице.

Для выполнения достаточно находиться на том же источнике (**домен**/**протокол**/**порт**), при этом **URL-путь** может быть разным.

Один источник:

```
http://js/test/index.html
```

```
http://js/target/example.html
```



Разные источники:

http://js/test/index.html

http://php-course/test/index.html

example_6-1. Передача данных хранилища localStorage между страницами

```
<script>
    // JS-объект данных
    let user = {
        name: "Евгений",
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Ленинград",
            post: 101101
        },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
        ]
    }
    // JSON-представление объекта
    userJSON = JSON.stringify(user);
    // сохраним представление в localStorage
    window.localStorage.setItem("user", userJSON);
    // JSON-объект, который получим на странице example_6-2.html
    console.log("Предварительный вывод объекта JSON:");
```

```
    console.log(localStorage.getItem("user"));
</script>
```

Получим данные на другой странице.

example_6-2. Получение данных

```
<script>
    // получим JSON-представление объекта из localStorage
    let userJSON = localStorage.getItem('user');
    console.log("Объект создан на странице example_6-1.html:");
    // объект JSON
    console.log(userJSON);
    // объект JS
    console.log(JSON.parse(userJSON));
</script>
```

Объект **localStorage** доступен всем окнам из одного источника, поэтому, если мы устанавливаем данные в одном окне, изменения становятся видимыми в другом.

В следующем демонстрационном примере обратите внимание на атрибут **target** ссылки:

```
<a href="example_7-2.html" target="_blank">Тестируем передачу объекта в localStore</a>
```

example_7-1. Передача данных хранилища localStorage между вкладками

```
<script>
    // JS-объект данных
    let user = {
        name: "Евгений",
```

```
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Ленинград",
            post: 101101
    },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
        ]
    }
    // преобразуем JS в JSON
    userJSON = JSON.stringify(user);
    // сохраним строку JSON в localStorage
    window.localStorage.setItem("user", userJSON);
    // JSON-объект, который получим на странице example_7-2.html
    console.log("Предварительный вывод объекта:");
    console.log(localStorage.getItem("user"));
</script>
```

Получаем данные на другой вкладке браузера.

example_7-2. Получение данных

```
<script>
    // получаем информацию из localStorage
    let userJSON = localStorage.getItem('user');
    // объект JSON
```

```
    document.write("<b>строка JSON:</b> ", userJSON, "<p>"); //
    {"name":"Евгений","surname": ...

    // объект JS

    document.write("<b>объект JS:</b> ", JSON.parse(userJSON));
    // [object Object]
</script>
```

В следующем демонстрационном примере протестируем передачу данных объектом хранилища **localStorage** между разными источниками.

Важно. При тестировании демонстрационных примеров **example_8** разместите файлы сценариев на **разных доменах**. В моем случае файлы располагаются:

Домен **js**:

- example_8-1
- example_8-2

Домен **php-course**:

- example_8-3

example_8-1. Передача данных хранилища localStorage между доменами

```
<script>

    let test = "Когда дела пойдут не так, эти цивилизованные люди
    сожрут друг друга (Джокер)";

    // сохранение строки в localStorage

    window.localStorage.setItem('key', test);

</script>
```

example_8-2. Получение данных

```
<script>
    // этот файл лежит на домене js
    // получение данных localStorage по ключу
    let test = localStorage.getItem('key');
    // вывод данных в браузер
    document.write("<h2>" + test + "</h2>");
</script>
```

example_8-3. Получение данных

```
<script>
    // этот файл лежит на домене php-course
    // получение данных localStorage по ключу
    let test = localStorage.getItem('key');
    // вывод данных в браузер
    document.write("<h2>" + test + "</h2>");
</script>
```

Как правило, сохраняемый в веб-хранилище объект – это результат каких-либо вычислений или обработки данных. Очень часто это данные, полученные с сервера, об этом будем говорить в соответствующей теме.

example_9-1. Данные веб-хранилище всегда результат обработки данных

```
<script>
    // очистим хранилище
    localStorage.clear();
    let id_course = 4;
```

```
    // поиск курса с идентификатором id_course = 4
    course = courses.find( item => {
        return item.id == id_course;
    });
    // console.log(course);
    // поиск всех ведущих курса
    persons = leaders.filter( item => {
        return item.courses.includes(id_course);
    });
    // console.log(persons);
    // формирование JS-объекта данных
    let data = {
        course: course,
        persons: persons
    };
    // JS в JSON
    dataJSON = JSON.stringify(data);
    // сохраним строку JSON в хранилище localStorage
    localStorage.setItem("data", dataJSON);
</script>
```

example_9-2. Получение данных

```
<script>
    // получение данных объекта localStorage по ключу
    let dataJSON = localStorage.getItem("data");
    // JS в JSON
```

```
    data = JSON.parse(dataJSON);

    // вывод объекта JS в консоль

    console.log(data);

</script>
```

sessionStorage

Объект **sessionStorage** используется гораздо реже, чем **localStorage**.

Свойства и методы такие же, но есть существенные ограничения:

- **sessionStorage** существует только в рамках **текущей вкладки браузера**.
    1. Другая вкладка с той же страницей будет иметь другое хранилище.
    2. Но оно разделяется между ифреймами на той же вкладке (при условии, что они из одного и того же источника).
- Данные продолжают существовать после перезагрузки страницы, но не после закрытия/открытия вкладки.

Протестируем **sessionStorage** на практике.

Если запустить следующий код:

```
sessionStorage.setItem('test', "I'm Sylvia");
```

… и обновить страницу, данные всё ещё можно получить:

```
alert(sessionStorage.getItem('test')); // I'm Sylvia
```

Но если вы откроете ту же страницу в другой вкладке и попробуете получить данные снова, то код выше вернёт **null**, что значит "ничего не найдено".

Так получилось, потому что **sessionStorage** привязан не только к источнику, но и к **вкладке браузера**. Поэтому **sessionStorage** используется нечасто.

example_10-1. Передача данных хранилища sessionStorage между страницами

```
<script>
    // JS-объект данных
    let user = {
        name: "Евгений",
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Ленинград",
            post: 101101
        },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
        ]
    }
    // преобразуем JS в JSON
    userJSON = JSON.stringify(user);
    // сохраняем JSON в sessionStorage
    window.sessionStorage.setItem("user", userJSON);
    // JSON-объект, который получим на странице example_10-2.html
    console.log("Предварительный вывод объекта:");
    console.log(sessionStorage.getItem("user"));
</script>
```

Получим данные на другой странице.

example_10-2. Получение данных

```
<script>
    // получение значения хранилища sessionStorage
    let userJSON = sessionStorage.getItem('user');
    // объект JSON
    document.write("<b>Строка JSON:</b> ", userJSON, "<p>");
    // объект JS
    document.write("<b>Объект JS:</b> ", JSON.parse(userJSON));
</script>
```

example_11-1. Передача данных хранилища sessionStorage между вкладками

```
<script>
    // JS-объект данных
    let user = {
        name: "Евгений",
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Ленинград",
            post: 101101
        },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
        ]
```

```
    }
    // преобразуем JS в JSON
    userJSON = JSON.stringify(user);
    // сохраняем JSON в sessionStorage
    window.sessionStorage.setItem("user", userJSON);
    // JSON-объект, который получим на странице example_11-2.html
    console.log("Предварительный вывод объекта:");
    console.log(sessionStorage.getItem("user"));
</script>
```

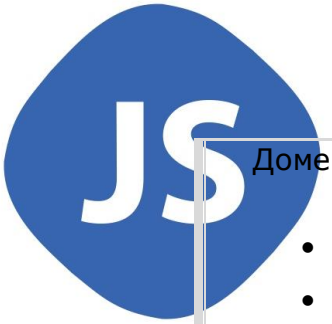
Получим данные на другой вкладке браузера.

example_11-2. Получение данных

```
<script>
    // получение значения хранилища sessionStorage
    let userJSON = sessionStorage.getItem('user');
    // объект JSON
    document.write("<b>Строка JSON:</b> ", userJSON, "<p>");
    // объект JS
    document.write("<b>Объект JS:</b> ", JSON.parse(userJSON));
</script>
```

В следующем демонстрационном примере протестируем передачу данных объектом хранилища **sessionStorage** между разными источниками.

Важно. При тестировании демонстрационных примеров **example_12** разместите файлы сценариев на **разных доменах**. В моем случае файлы располагаются:



Домен **js**:

- example_12-1
- example_12-2

Домен **php-course**:

- example_12-3

example_12-1. Передача данных хранилища sessionStorage между доменами

```
<script>
    let test = "Мне нужны новые ненавистники, старые стали моими
    фанатами (Джокер)";
    // сохранение строки в sessionStorage
    window.sessionStorage.setItem('key', test);
</script>
```

example_12-2. Получение данных

```
<script>
    // этот файл лежит на домене js
    // получение данных sessionStorage по ключу
    let test = sessionStorage.getItem('key');
    // вывод данных в браузер
    document.write("<h2>" + test + "</h2>");
</script>
```

example_12-3. Получение данных

```
<script>
```



```
    // этот файл лежит на домене php-course
    // получение данных sessionStorage по ключу
    let test = sessionStorage.getItem('key');
    // вывод данных в браузер
    document.write("<h2>" + test + "</h2>");
</script>
```

Перебор ключей

Методы работы с объектами веб-хранилищ позволяют читать/писать/удалять данные. Необходимо рассмотреть, каким образом можно получить все **значения** или **ключи** хранилища. К сожалению, объекты веб-хранилища нельзя перебрать в цикле, они **не итерируемы**.

Но, если с объектами веб-хранилищ можно работать как с обычными объектами JavaScript, то можно пройти по ним, как по **обычным массивам**. Используем для этого метод **key**() – получение ключа на заданной позиции.

example_13. Перебор ключей веб-хранилища циклом for

```
<script>
    // JS-объект
    let user = {
        name: "Евгений",
        surname: "Лукашин",
        address: {
            street: "3-я улица Строителей, дом 25",
            city: "Печора",
            post: 101101
        },
        phone: [
            "(912)552-92-25",
            "(904)104-17-06"
```

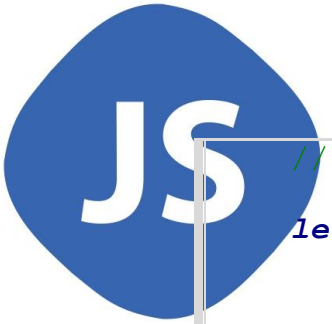
```
        ],
        edu: [
            "Даугавпилсское высшее военное авиационное инженерное
            училище",
            "Ивановский энергетический университет"
        ]
    }
    // очистим хранилище
    localStorage.clear();
    // сохраним ключи JS объекта в объект хранилища
    localStorage.setItem("name", user.name);
    localStorage.setItem("surname", user.surname);
    localStorage.setItem("address",
    JSON.stringify(user.address));
    localStorage.setItem("phone", JSON.stringify(user.phone));
    localStorage.setItem("edu", JSON.stringify(user.edu));
    // пройдем циклом по ключам хранилища
    for(let i=0; i<localStorage.length; i++) {
        // метод key - получим ключ на заданной позиции
        let key = localStorage.key(i);
        // значения ключей выводятся в виде строк
        console.log(key, " -> ", localStorage.getItem(key));
    }
</script>
```

Другой способ – использовать цикл, как по **обычному объекту** с помощью специального цикла **for...in**.

В следующем демонстрационном примере перебираются ключи, но вместе с этим выводятся несколько **встроенных полей объекта**, которые нам не нужны.

example_14. Перебор ключей веб-хранилища специальным циклом for...in

```
<script>
```



```
// JS-объект
let user = {
    name: "Евгений",
    surname: "Лукашин",
    address: {
        street: "3-я улица Строителей, дом 25",
        city: "Печора",
        post: 101101
    },
    phone: [
        "(912)552-92-25",
        "(904)104-17-06"
    ],
    edu: [
        "Даугавпилсское высшее военное авиационное инженерное училище",
        "Ивановский энергетический университет"
    ]
}
// очистим хранилище
localStorage.clear();
localStorage.setItem("name", user.name);
localStorage.setItem("surname", user.surname);
localStorage.setItem("address", JSON.stringify(user.address));
localStorage.setItem("phone", JSON.stringify(user.phone));
localStorage.setItem("edu", JSON.stringify(user.edu));
```



```
    // специальный цикл for
    for(key in localStorage) {
        // значения ключей выводятся в виде строк
        console.log(key, " -> ", localStorage.getItem(key));
    }
</script>
```

Для того, чтобы отфильтровать встроенные поля объекта необходимо воспользоваться проверкой **hasOwnProperty** (рассмотрим в теме, посвященной объектам).

Другой способ получить **собственные ключи** объекта с помощью **Object.keys**, а затем при необходимости вывести их при помощи цикла.

example_15. Перебор массива ключей веб-хранилища специальным циклом for...of

```
<script>
    // очистим хранилище
    localStorage.clear();
    // сохраним объекты массива в localStorage
    personnel.forEach( (item, key) => {
        itemJSON = JSON.stringify(item);
        localStorage.setItem(key, itemJSON);
    });
    // получим набор ключей объекта localStorage
    let keys = Object.keys(localStorage);
    // тестовый вывод
    // console.log(keys.sort());
    // специальный цикл по массиву ключей
    for(let key of keys.sort()) {
```

```
        // извлекаем значение ключа
        itemJSON = localStorage.getItem(key);
        // преобразуем строку JSON в JS объект
        item = JSON.parse(itemJSON);
        // выводим в консоль пару ключ -> значение
        console.log(key, " -> ", item);
    }
</script>
```

Последний пример работает, потому что Object**.keys** возвращает только ключи, принадлежащие объекту, игнорируя **встроенные поля объекта** (прототип объекта).

Событие Storage

Когда **обновляются данные** в **localStorage** или **sessionStorage**, генерируется событие **storage** со следующими свойствами:

- **key** – ключ, который обновился (null, если вызван метод clear()).
- **oldValue** – старое значение (null, если ключ добавлен впервые).
- **newValue** – новое значение (null, если ключ был удалён).
- **url** – url документа, где произошло обновление.
- **storageArea** – объект **localStorage** или **sessionStorage**, где произошло обновление.

Примечание. Подробно о событиях в теме: "События и обработка событий"



Послесловие

Объекты веб-хранилища **localStorage** и **sessionStorage** позволяют хранить пары ключ/значение в браузере.

- **key** и **value** должны быть строками.
- Лимит 5 Мб+, зависит от браузера.
- Данные не имеют **времени** истечения.
- Данные **привязаны** к источнику (домен/протокол/порт).

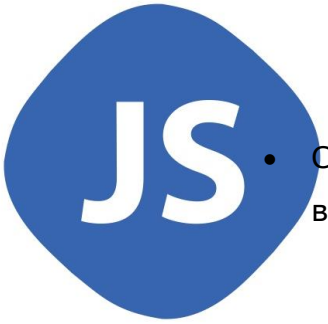
localStorage	**sessionStorage**
Совместно используется между всеми вкладками и окнами с одинаковым источником	Разделяется в рамках вкладки браузера, среди ифреймов из того же источника
Переживает перезапуск браузера	Переживает перезагрузку страницы (но не закрытие вкладки)

Методы и **свойства** объектов веб-хранилища:

- **setItem**(key, value) – сохранить пару ключ/значение.
- **getItem**(key) – получить данные по ключу key.
- **removeItem**(key) – удалить значение по ключу key.
- **clear**() – удалить всё.
- **key**(index) – получить ключ на заданной позиции.
- **length** – количество элементов в хранилище.
- Используйте Object.**keys** для получения всех ключей.
- Можно обращаться к ключам как к обычным свойствам объекта, в этом случае событие **storage** не срабатывает.

В **самостоятельных работах** к уроку рассмотрим:

- Поиск в массиве и сохранение в веб-хранилище требуемого объекта.
- Создание из связанных массивов сложной структуры данных, сохранение в веб-хранилище.
- Передача сложных структур данных между страницами.



- Создание и сохранение в хранилище структуры данных из дерева DOM, вывод структуры в документ.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы

